

Module 4: Updating Items

 web.archive.org/web/20230529002703/https://aws.amazon.com/getting-started/hands-on/create-manage-nonrelational-database-dynamodb/module-4

Getting Started with AWS

Create and Manage a Nonrelational Database

with Amazon DynamoDB

You will use the `UpdateItem` API to update attributes of an existing item in our database

Overview

In this module, you use the *UpdateItem* API to update attributes of an existing item in our database

Application use cases

In addition to writing and reading data, you regularly want to update existing data in a database. It's best if you can update this data in a single call rather than first reading an item and then writing back the full item with our given updates. DynamoDB allows you to update items in place by using the *UpdateItem* API call.

For this example, recall that each item has a *Formats* attribute, which is a map of all the different formats your bookstore has for a given title. Over time, this map needs to be updated for a specific book, either because you have added a new format or because you stopped carrying a format.

Time to complete

15 minutes

Implementation

Updating an item

When updating an item in your application code, focus on four elements:

- The *Key* that identifies the item you want to update.
- Expression attribute names, which are placeholders for the attributes you want to update.
- Expression attribute values, which are placeholders for the values to which you want to update your attributes.
- The update expression, which uses the expression attribute names and expression attribute values to declare the updates you want. Imagine that you have recently added an audiobook format for John Grisham's *The Rainmaker*, and you want to update our DynamoDB table to reflect that change. The *UpdateItem* API call would look as follows.

```
# The UpdateItem API allows you to update a particular item as identified by its key.
resp = table.update_item(
    Key={"Author": "John Grisham", "Title": "The Rainmaker"},
    # Expression attribute names specify placeholders for attribute names to use in your u
    ExpressionAttributeNames={
        "#formats": "Formats",
        "#audiobook": "Audiobook",
    },
    # Expression attribute values specify placeholders for attribute values to use in your
    ExpressionAttributeValues={
        ":id": "8WE3KPTP",
    },
    # UpdateExpression declares the updates you want to perform on your item.
    # For more details about update expressions, see https://docs.aws.amazon.com/amazondyn
    UpdateExpression="SET #formats.#audiobook = :id",
)
```

Your *UpdateItem* API call has each of the four elements. First, it specifies the *Key* of the item it wants to operate on by providing the *Author* as *John Grisham* and the *Title* as *The Rainmaker*.

Second, you specify the expression attribute names that you will use in your update expression. You are modifying the *Formats* attribute, which is a map, so you need to define expression attribute names for both the attribute (*Formats*) and the key within your map (*Audiobook*).

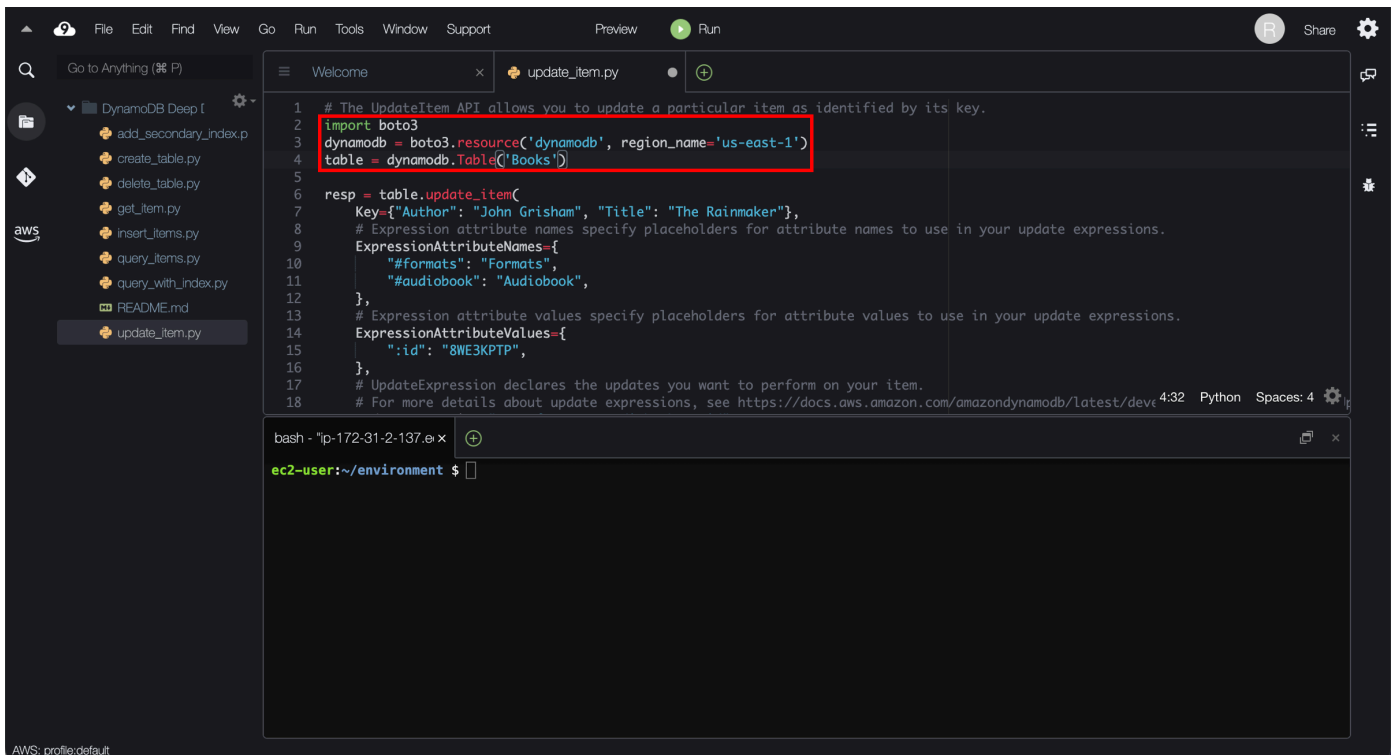
Finally, you write the update expression that you want to apply in your *UpdateItem* API call. This update expression uses both the expression attribute names and expression attribute values as previously defined. Your expression of *SET #formats.#audiobook = :id* tells DynamoDB to set the *Audiobook* key of the *Formats* attribute to the given ID value of *8WE3KPTP*.

Third, you state the value you want to use in the expression attribute values. In your *Formats* map, the value for a format is the item number in your inventory system. In this example, the value is *8WE3KPTP*.

The *update_item.py* script makes the *UpdateItem* API call shown in the code block earlier in the module. To demonstrate the difference made by the call, the script first retrieves and prints the

item before the update call is made. It then makes the *UpdateItem* API call. Finally, the script retrieves and prints the item a second time to show the difference.

You need to add a few lines of code before you run the script. Open *update_item.py* in Cloud9, and paste the following three lines at the top of the script:



```
1 # The UpdateItem API allows you to update a particular item as identified by its key.
2 import boto3
3 dynamodb = boto3.resource('dynamodb', region_name='us-east-1')
4 table = dynamodb.Table('Books')
5
6 resp = table.update_item(
7     Key={"Author": "John Grisham", "Title": "The Rainmaker"},
8     # Expression attribute names specify placeholders for attribute names to use in your update expressions.
9     ExpressionAttributeNames={
10         "#formats": "Formats",
11         "#audiobook": "Audiobook",
12     },
13     # Expression attribute values specify placeholders for attribute values to use in your update expressions.
14     ExpressionAttributeValues={
15         ":id": "8WE3KPTP",
16     },
17     # UpdateExpression declares the updates you want to perform on your item.
18     # For more details about update expressions, see https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/UpdateExpressions.html
```

```
import boto3
dynamodb = boto3.resource('dynamodb', region_name='us-east-1')
table = dynamodb.Table('Books')
```

Next, run the following command:

```
$ python update_item.py
```

Now, run the following command:

```
$ python get_item.py
```

You should see the following output:

```
{'Title': 'The Rainmaker', 'Formats': {'Audiobook': '8WE3KPTP', 'Hardcover': 'J4SUKVGU', 'Paperback': 'J4SUKVGU', 'Unknown': 'J4SUKVGU'}}
```

As you can see, *The Rainmaker* has a new *Audiobook* format after the update has been applied.

In this module, you learned how to update an existing item with the *UpdateItem* API. In the next module, you learn how to clean up the resources you created to avoid additional charges.